HALMSTAD UNIVERSITY

Computer Systems Engineering I

Software Lecture I

Hazem Ali

HALMSTAD UNIVERSITY

Learning Objectives

After this lecture you can:



Explain how C differs from Java, and why C is used on microcontrollers



Describe how a C program becomes code running on the board



Choose the right data type, including the fixed-width types in <stdint.h>



Set, clear, toggle and test bits with C operators



Split a program into .c and .h files

After this lecture you can start working on Lab 1.

HALMSTAD UNIVERSITY

Outline

1. Embedded programming
2. From Java to C
3. Types and variables
4. Operators and bits
5. Control flow and functions
6. Programs in several files

HALMSTAD UNIVERSITY

Embedded Programming

Part 1 of 6

HALMSTAD UNIVERSITY

Embedded Programming



You must know the hardware

Datasheets, schematics and manuals are your documentation.



Usually no operating system

“Bare metal”: your program is the only software on the chip.



Limited resources

Kilobytes of memory, no floating-point unit, a clock in MHz rather than GHz.



Time matters

A correct value delivered too late is a wrong value.

Languages: C dominates. Assembler is used for start-up code and time-critical parts; C++ and Rust are growing.

HALMSTAD UNIVERSITY

The SAM3X8E

SAM3X8E microcontroller (on the Arduino Due)

Cortex-M3 CPU
up to 84 MHz

Flash
512 KB

SRAM
96 KB

Bus: every peripheral is reached through registers at fixed addresses

PIO A–D
GPIO pins

Timers
9 channels

PWM
8 channels

ADC
12-bit

UART / USART
serial

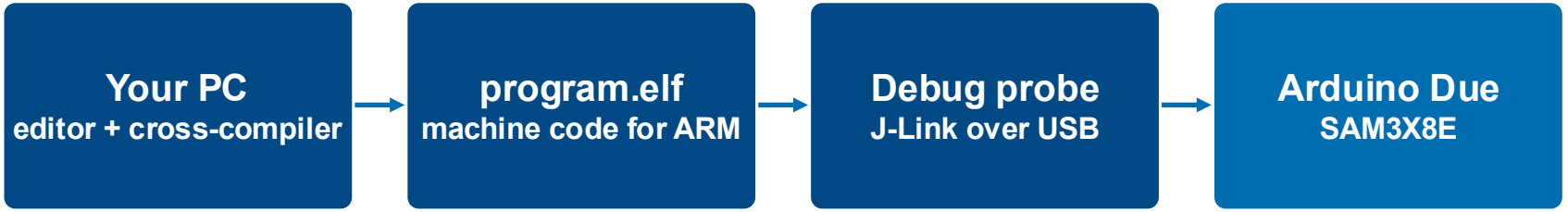
PMC
clocks

On the lab board

- LEDs and button
- Keypad
- Display
- Temperature sensor
- Light sensor
- Servo motor

HALMSTAD UNIVERSITY

Tools and Workflow



A cross-compiler runs on your PC but produces machine code for another processor: here the ARM Cortex-M3.

In the lab

- IAR Embedded Workbench

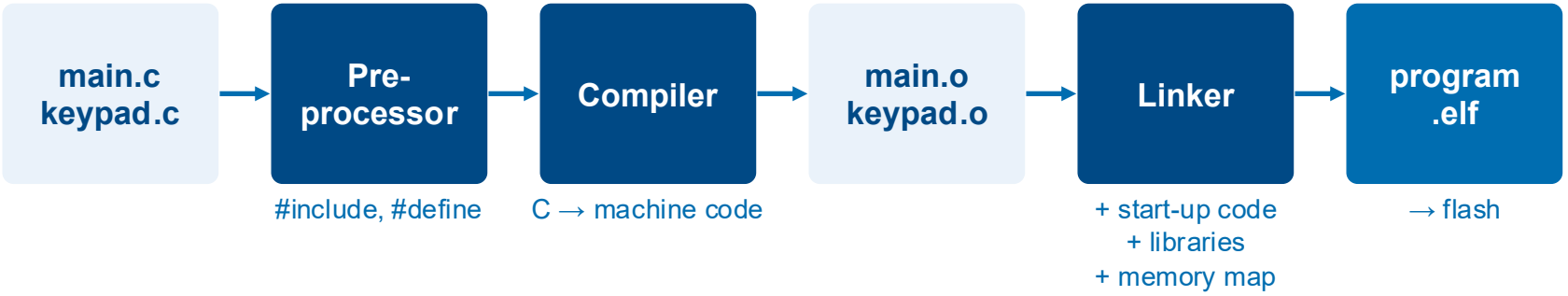
At home (free tools)

- Eclipse Embedded CDT
- Arm GNU Toolchain (GCC)
- SEGGER J-Link drivers

Installation guides for the home tools are on Blackboard.

HALMSTAD UNIVERSITY

The Build Process




```
arm-none-eabi-gcc -mcpu=cortex-m3 -mthumb -c main.c -o main.o
```

Compile error

A problem inside one file: a missing ;, an unknown type, a wrong argument.

```
main.c:12: error: expected ';'
```

Link error

Each file compiled, but a function was declared and never defined, or the names differ.

```
undefined reference to `calculateSum'
```

HALMSTAD UNIVERSITY

From Java to C

Part 2 of 6

HALMSTAD UNIVERSITY

C vs Java

	Java	C
Program structure	Classes and objects	**Functions and data – no classes**
Memory	new + garbage collector	**You manage it: static, stack (heap rarely)**
Errors	Exceptions	**Return values – no exceptions**
Arrays	Know their length, bounds checked	**No length, no bounds check**
Strings	String class	**char arrays ending with '\0'**
Uninitialised locals	Compile error	**Contain garbage**
Integer sizes	Fixed (int = 32 bits)	**Depend on the platform**
Unsigned types	None (except char)	**unsigned char, int, long …**
Pointers	References only	**Any address, with arithmetic**

HALMSTAD UNIVERSITY

Undefined Behaviour

C trusts you. Many mistakes are not errors: the behaviour is undefined – anything can happen, and nothing tells you.

```
int a[4];
a[4] = 7;                /* writes past the end of the array */

int x;
int y = x + 1;           /* x was never set: any value */

int big = 2147483647;
big = big + 1;           /* signed overflow: undefined */
```



On a microcontroller there is no crash message. You may silently overwrite another variable, the stack or a hardware register. Compile with all warnings on (`-Wall -Wextra`) and read them.

HALMSTAD UNIVERSITY

main() on the Board

```
int main(void)
{
    init_hardware();        /* clocks, pins, interrupts */

    while (1) {             /* the "superloop" never ends */
        read_inputs();
        update_outputs();
    }
}
```

- On a PC, main() returns to the operating system. On the board there is no operating system, so main() never returns.
- Before main() runs, start-up code sets up the stack and copies initial values into RAM (more in the HW lectures).
- Your project will have exactly this shape: initialise once, then loop forever.

HALMSTAD UNIVERSITY

Types and Variables

Part 3 of 6

HALMSTAD UNIVERSITY

Reserved Words

auto	break	case	char
const	continue	default	do
double	else	**enum**	**extern**
float	for	goto	if
int	long	register	return
short	signed	**sizeof**	**static**
struct	switch	**typedef**	**union**
unsigned	void	**volatile**	while

Especially important in embedded code

C99 added `inline`, `restrict` and `_Bool` (use `bool` from `<stdbool.h>`).

HALMSTAD UNIVERSITY

Scope and Lifetime

```
int count = 0;            /* global: usable from other files */
static int hidden;        /* file scope: only this file, starts at 0 */

void tick(void)
{
    int local = 5;        /* new copy on every call (stack) */
    static int calls;     /* keeps its value between calls */

    calls++;
    count += local;
}
```

- **Scope:** where a name can be used. **Lifetime:** how long the variable exists.
- A declaration says a name exists (`extern int count;`); a definition creates it (`int count = 0;`).
- **Local variables are not zeroed: always initialise them.**

HALMSTAD UNIVERSITY

Integer Types

Type	Bytes on Cortex-M3	Bytes on 64-bit PC	Range on Cortex-M3
`char`	1 (unsigned)	1 (signed)	0 … 255
`signed char`	1	1	−128 … 127
`short`	2	2	−32 768 … 32 767
`unsigned short`	2	2	0 … 65 535
`int`	4	4	−2 147 483 648 … 2 147 483 647
`unsigned int`	4	4	0 … 4 294 967 295
`long`	4	8 *	same as int
`long long`	8	8	about $\pm 9.2 \times 10^{18}$
`pointer`	4	8	–

** 8 bytes on Linux and macOS, 4 on Windows. Sizes depend on the platform – so use the fixed-width types on the next slide.*

HALMSTAD UNIVERSITY

Fixed-Width Types

```
#include <stdint.h>
#include <stdbool.h>

uint8_t  byte    = 0xFFu;      /* exactly 8 bits, 0 … 255 */
int16_t  temp    = -40;        /* exactly 16 bits, signed */
uint32_t mask    = 1u << 27;   /* exactly 32 bits */
bool     pressed = false;      /* true or false */
```

Use them for:

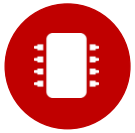
Hardware registers	**`uint32_t`**
Bytes on a data bus	**`uint8_t`**
Values that can be negative	**`int16_t`**, **`int32_t`**
Flags	**`bool`**

HALMSTAD UNIVERSITY

Floating-Point Types

Type	Bytes	Range	Precision
`float`	4	$\pm 1.2 \cdot 10^{-38}$ … $\pm 3.4 \cdot 10^{38}$	about 7 digits
`double`	8	$\pm 2.2 \cdot 10^{-308}$ … $\pm 1.8 \cdot 10^{308}$	about 15–16 digits
`long double`	8 *	as double on ARM	as double on ARM

** 16 bytes on a 64-bit PC.*



The Cortex-M3 has no floating-point unit. Every float or double operation runs in software: much slower than integer maths, and more code.

- Write `2.5f`, not `2.5` – a literal without f is a double.
- Prefer integers: store 23.45 °C as `2345` hundredths of a degree.

HALMSTAD UNIVERSITY

Constants and Enums

```
#define LED_PIN    27u                /* text replacement, no type */
const uint32_t BAUD_RATE = 9600u;     /* typed and read-only */

enum month { JAN = 1, FEB, MAR, APR, MAY, JUN,
             JUL, AUG, SEP, OCT, NOV, DEC };

enum state { IDLE, MEASURING, SHOWING };   /* 0, 1, 2 */
enum state current = IDLE;
```

`#define`	Replaced by the preprocessor before compiling. No type, no memory.
`const`	A real variable the compiler protects. A const table can stay in flash.
`enum`	Named integer constants – ideal for the states of your program.

HALMSTAD UNIVERSITY

Type Conversions

```
int   apples = 5;
float result;

result = apples / 2;          /* 2.0: integer division first */
result = apples / 2.0f;       /* 2.5: apples converted to float */
result = (float)apples / 2;   /* 2.5: explicit cast */

int      i = -1;
unsigned u = 1u;
if (i < u) { }                /* false! i becomes 0xFFFFFFFF */
```



- Integer division throws away the fraction: `5 / 2` is 2.
- **Mixing signed and unsigned converts the signed value to unsigned.**
- Compile with `-Wall -Wextra` to get a warning for both.

HALMSTAD UNIVERSITY

Storage Classes

Storage classes

- **`auto`** – the default for local variables; never written.
- **`register`** – an old hint; modern compilers ignore it.
- **`static`** – at file level: private to the file. Inside a function: keeps its value between calls.
- **`extern`** – declared here, defined in another file.

Type qualifiers

- **`const`** – read-only.
- **`volatile`** – every read and write really goes to memory; the compiler may not keep the value in a CPU register or remove the access.
- **Needed for hardware registers and for variables shared with interrupts (SW2).**

HALMSTAD UNIVERSITY

Operators and Bits

Part 4 of 6

HALMSTAD UNIVERSITY

Operators

Group	Operators
Arithmetic	`+ - * / %`
Comparison	`== != < > <= >=`
Logical	`&& || !`
Increment	`++ --`
Assignment	`= += -= *= /= &= |= ^= <<= >>=`
Bitwise	`~ &	^ << >>` (next slide)

```
i = 4; j = 7;
a = ++i * --j;        /* 5 * 6 = 30 */
i = 4; j = 7;
a = i++ * j--;        /* 4 * 7 = 28 */
```

- Same as Java, but C has no >>> operator.
- A condition is any integer: 0 is false, everything else is true.
- `!!i == i` is only true when i is 0 or 1.

HALMSTAD UNIVERSITY

Bitwise Operators

Example: **`uint8_t c = 0x8F;`** which is **`1000 1111`**

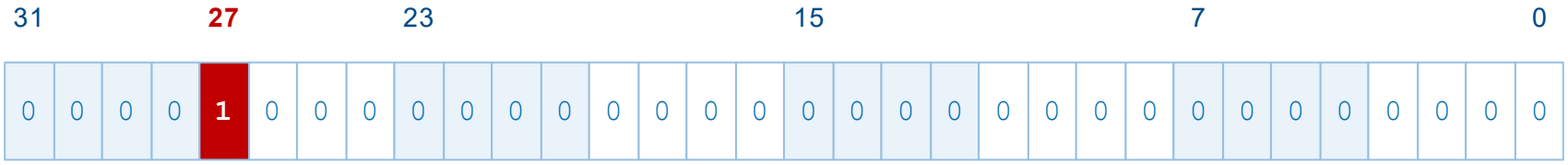
Operator	Name	Expression	Result (binary)	Hex
~	NOT	`~c`	`0111 0000`	`0x70`
&	AND	`c & 0x0F`	`0000 1111`	`0x0F`
|	OR	`c | 0x30`	`1011 1111`	`0xBF`
^	XOR	`c ^ 0xFF`	`0111 0000`	`0x70`
<<	Shift left	`1u << 4`	`0001 0000`	`0x10`
>>	Shift right	`c >> 2`	`0010 0011`	`0x23`

Note: `~c` is computed as an int (0xFFFFFF70); stored back in a `uint8_t` it becomes 0x70.

HALMSTAD UNIVERSITY

Why Bits Matter

One 32-bit register controls 32 pins of port B. Each bit is one pin.



PB27 → the LED on pin 13

```
/* PIOB Set Output Data Register: address from the datasheet */
#define PIOB_SODR  (*(volatile uint32_t *)0x400E1030u)

PIOB_SODR = 1u << 27;      /* drive PB27 high: the LED lights */
```

- Writing a 1 to bit 27 of this register sets pin PB27 high; the other 31 pins are not affected.
- The pin must first be configured as an output (HW Lecture 1).
- How the #define line works – volatile pointers to fixed addresses – is the topic of SW2.

Bit Manipulation Idioms

```
#define BIT(n)    (1u << (n))

reg |=  BIT(5);             /* set bit 5 */
reg &= ~BIT(5);             /* clear bit 5 */
reg ^=  BIT(5);             /* toggle bit 5 */

if (reg & BIT(5))     { }   /* is bit 5 set? */
if (!(reg & BIT(9))) { }    /* is bit 9 clear? */
```

`|=`	`&= ~`	`^=`	`&`
sets bits	clears bits	toggles bits	tests bits

- Bits are numbered from 0, as in the datasheet: “bit 5” is the sixth bit.
- Always use unsigned values (`1u`, `uint32_t`) for bit operations.

HALMSTAD UNIVERSITY

Multi-Bit Fields



MODE field: bits 6…4

```
#define MODE_POS    4u
#define MODE_MASK   (0x7u << MODE_POS)        /* bits 6..4 */

/* read the field */
mode = (reg & MODE_MASK) >> MODE_POS;

/* write it: clear the field, then insert the new value */
reg = (reg & ~MODE_MASK) | ((mode << MODE_POS) & MODE_MASK);

/* are bits 11 and 12 both 1? */
if ((reg & (3u << 11)) == (3u << 11)) { }
```

This read–modify–write pattern changes a few bits without touching the others. The SAM3X avoids it for pins by giving you separate set and clear registers (HW Lecture 1).

HALMSTAD UNIVERSITY

Bit Manipulation Pitfalls

Instead of

```
k &= 0xFFDF;
```

On 32 bits this also clears bits 16–31.

Write

```
k &= ~BIT(5);
```

Instead of

```
x = 1 << 31;
```

Overflows a signed int: undefined.

Write

```
x = 1u << 31;
```

Instead of

```
if (reg & MASK == MASK)
```

== binds tighter than &.

Write

```
if ((reg & MASK) == MASK)
```

Instead of

```
s >> 2    /* s < 0 */
```

Shifting a negative value: compiler-specific.

Write

```
u >> 2    /* u unsigned */
```

HALMSTAD UNIVERSITY

Try It

```
uint32_t reg = 0xF0u;    /* 1111 0000 */

reg |=  BIT(1);
reg &= ~BIT(4);
reg ^=  BIT(7);
```



- What is the value of reg now, in binary and in hex?
- Write one line that clears bits 0–3 of reg and leaves the rest unchanged.
- Write a test that is true only when bits 4 and 7 are both 1.

HALMSTAD UNIVERSITY

Control Flow and Functions

Part 5 of 6

HALMSTAD UNIVERSITY

Control Flow

if / else

```
if (n > 0) {
    avg = sum / n;
} else if (n == 0) {
    avg = 0;
} else {
    error = true;
}
```

switch

```
switch (color) {
case YELLOW:
    yellow++;
    break;
case BLUE:
    blue++;
    break;
default:
    others++;
    break;
}
```

- Same as Java. A condition is any integer: 0 is false, anything else is true.
- Forget `break` and the switch falls through to the next case.
- **Classic bug:** `if (x = 5)` assigns 5 and is always true. Write `==`.

HALMSTAD UNIVERSITY

Loops

for / while

```
/* n! with for */
fac = 1;
for (i = 2; i <= n; i++) {
    fac *= i;
}

/* the same with while */
fac = 1;
i = 1;
while (++i <= n) {
    fac *= i;
}
```

do-while / forever

```
/* wait until the display
   reports ready */
do {
    status = read_status();
} while ((status & 0x03u)
         != 0x03u);

/* loop forever */
while (1) {
    /* ... */
}
```

- Since C99 you can write `for (int i = 0; i < n; i++)`.
- `do-while` always runs at least once – useful for “read, then check”.

HALMSTAD UNIVERSITY

Arrays

```
#define N 100u

int32_t samples[N];
int32_t sum = 0;

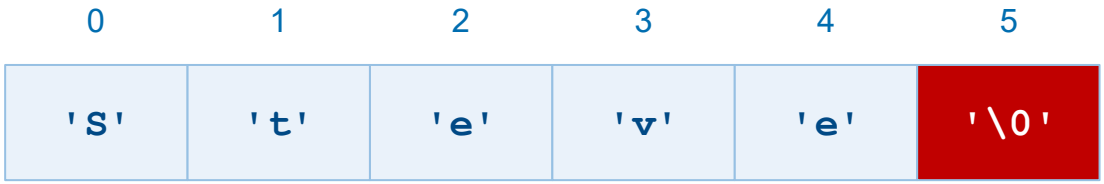
for (uint32_t i = 0; i < N; i++) {
    sum += samples[i];
}
int32_t avg = sum / (int32_t)N;     /* divide once, at the end */

int matrix[3][3] = { {1, 2, 3}, {4, 5, 6}, {7, 8, 9} };
size_t count = sizeof samples / sizeof samples[0];   /* 100 */
```

- No `.length`: keep the size in a constant, or compute it with `sizeof`.
- **No bounds check:** `samples[100]` compiles – and overwrites whatever is stored next.
- An array is not an object: you cannot assign one array to another with `=`.

HALMSTAD UNIVERSITY

Strings



A string is a char array that ends with a zero byte.

```
#include <string.h>

char name[] = "Steve";              /* 6 bytes: 5 letters + '\0' */
char copy[16];

strncpy(copy, name, sizeof copy - 1);
copy[sizeof copy - 1] = '\0';       /* make sure it is terminated */

if (strcmp(name, "Steve") == 0) { }   /* not: name == "Steve" */
size_t len = strlen(name);           /* 5 */
```

- == compares addresses, not text: use `strcmp()`.
- Your display driver will send these bytes to the display one by one.

HALMSTAD UNIVERSITY

Functions

```c
/* prototype: usually in a .h file */
float calc_avg(float x, float y);

/* definition */
float calc_avg(float x, float y)
{
    return (x + y) / 2.0f;    /* 2.0f keeps the maths in float */
}

avg = calc_avg(a, b);         /* a and b are copied */
```

- **Arguments are passed by value:** the function works on copies.
- A function returns one value. To return more, use a struct or pointers (SW2).
- `static` before a function makes it private to its file.
- `void f(void)` means “no parameters”; `void f()` means “unspecified” in C.

HALMSTAD UNIVERSITY

Programs in Several Files

Part 6 of 6

HALMSTAD UNIVERSITY

Multi-File Programs

simplesum.h

```
#ifndef SIMPLESUM_H
#define SIMPLESUM_H

float calculate_sum(int a, float b);

#endif
```

simplesum.c

```
#include "simplesum.h"

float calculate_sum(int a, float b)
{
    return (float)a + b;
}
```

main.c (PC version)

```
#include <stdio.h>
#include "simplesum.h"

int main(void)
{
    float sum;

    sum = calculate_sum(1, 2.5f);
    printf("%f\n", sum);
    return 0;
}
```

- The header declares; the .c file defines; every user includes the header. The include guard stops double inclusion.
- C is case-sensitive: `calculateSum` and `CalculateSum` are different functions.

HALMSTAD UNIVERSITY

The Preprocessor

```
#include <stdint.h>         /* system header: <...> */
#include "keypad.h"         /* your own header: "..." */

#define N_KEYS          12u
#define SQUARE_BAD(x)   x * x
#define SQUARE(x)       ((x) * (x))
/* SQUARE_BAD(a + 1) becomes  a + 1 * a + 1   (wrong!) */

static inline uint32_t square(uint32_t x) { return x * x; }

#ifdef DEBUG
/* only compiled when DEBUG is defined */
#endif
```

- The preprocessor edits the text of your file before the compiler sees it.
- Put parentheses around every macro argument and around the whole macro – or use a static inline function.
- Use `true` / `false` from `<stdbool.h>` instead of your own TRUE and FALSE.

HALMSTAD UNIVERSITY

Summary

1 Embedded

Know the hardware; usually no OS; limited memory and no FPU.

2 Java → C

No classes, garbage collector, exceptions or bounds checks.

3 Types

Sizes depend on the platform: use <stdint.h>. Prefer integers.

4 Bits

|= sets, &= ~ clears, ^= toggles, & tests – always unsigned.

5 Control flow

Like Java. Arguments are copies; strings end with '\0'.

6 Files

Declarations in .h with an include guard, definitions in .c.

HALMSTAD UNIVERSITY

Review Questions

1. Name three differences between C and Java that can cause bugs.
2. What is the difference between a compile error and a link error?
3. Why use `uint32_t` instead of `int` for a hardware register?
4. Write code that sets, clears and tests bit 3 of `reg`.
5. What else does `k &= 0xFFDF;` change when k is 32 bits wide?
6. Why is `2.5f` a better choice than `2.5` on the Cortex-M3?
7. Why do we put prototypes in a header with an include guard?

HALMSTAD UNIVERSITY

Next Steps



Next lecture: SW2

Thursday 5 November – pointers, memory and hardware registers.



Lab 1

Register your group and book your lab sessions in the spreadsheet before Monday 9 November.



Practise

Try the bit exercises on your own PC with any C compiler.